2026

Project Management Guidebook

GROUP 3

ABDULWASIU ABDULLAHI,
ARPONEN MINNA,
ASONGACHA LUCAS,
AZIZ ULLAH MUHAMMAD,
EBERE EMMANUEL,
KALATHIYA DIYAJKUMAR

Contents

Chapter 1: Introduction 3
Chapter 2: Waterfall Project Management 3
2.1 The Waterfall framework & stages 3
2.2 Benefits & Limitations 4
2.3 When to use Waterfall 4
Chapter 3: Agile & Lean Project Management 5
3.1 Scrum Framework 5
3.2 User Stories & Scrum Boards 7
Introduction to the Scrum Board 8
3.3 Product Backlog & Sprint Backlog 10
Creating the Product Backlog 10
3.4 Other Agile Frameworks 12
Extreme Programming (XP) 12
Scaled Agile Framework (SAFe) 13
Feature-Driven Development (FDD) 13
Framework Comparison and Selection Guidance 14
3.5 Waterfall vs Agile – Choosing the right approach 14
Hybrid Approaches 15
Decision guides 17
Chapter 4: WBS & GANTT Chart 18
4.1 Work Breakdown Structure (WBS) 18
4.2 How to create a WBS 18
4.3 The GANTT Chart 20
Chapter 5: Scope Management 22
5.1 Defining Project Scope 22
5.2 Preventing Scope Creep 23
5.3 Key processes of scope management 24
5.4 How to make a scope management plan 24
Chapter 6: Risk Management 26
6.1 Identifying project risks 26
6.2 Risk assessment & analysis 26
4 27

6.3 Risk register & mitigation 29
Chapter 7: Change Management 30
7.1 What is Change Management 30
7.2 Change Management Plan 30
7.3 Change Request Process 32
Chapter 8: Quality Management 34
8.1 Quality in Projects 34
8.2 Quality Planning and Control 34
Agile vs. Waterfall: How Quality Management Differs 35
8.3 Quality tools and technique 37
Quick-Reference Guide: Choosing the Right Quality Tool 41
Chapter 9: PM Software 41
9.1 Popular Project Management tools 41
9.2 Choosing the right software 42
Chapter 10: Professional Development 43
10.1 PM Associations & Communities 43
10.2 Certifications 47
Chapter 11: Appendices 50
11.1 Source references 50
11.2 Ethical AI use statement 51
11.3 Ready-to-use templates 52
Template of Change Request Form 52

Chapter 1: Introduction

This Project Management Guidebook is meant to help any project team to understand different project management frameworks, project team roles, and responsibilities and how to choose the right management style to fit their project. The emphasis will be on SCRUM and Waterfall styles, but other frameworks will be introduced too. The guidebook will provide examples and templates whenever possible to help the reader understand the sometimes-complex concepts and terms. It also provides help to self-assess your skills and if you are fulfilling your role responsibilities.

Chapter 2: Waterfall Project Management

2.1 The Waterfall framework & stages

The Waterfall framework, first formally described by Royce (1970), is a step-by-step way of managing a project. You move through each phase in order, and you only start the next one once the previous stage is fully finished. Because of this, it works best when everything is clearly planned from the start and changes are unlikely.

How the Waterfall Model Works

Waterfall is built on a simple idea; plan everything early, follow a clear structure, and document each step properly. This makes it easy to track progress and know exactly where the project stands at any time. The downside is that going back to making changes later can be difficult.

Main Stages of Waterfall

1. Requirements
All project needs are gathered and written down. This is where you define what the final result should be.

2. Design
The requirements are turned into a clear plan, including how the system will look and work.

3. Implementation
The actual development happens here. The product is built based on the design.

4. Testing
The finished system is checked for errors and tested to make sure it meets the requirements.

5. Deployment
The product is delivered or launched for users.

6. Maintenance
After launch, any issues are fixed, and updates are made when needed.

2.2 Benefits & Limitations

Before choosing Waterfall for a project, it's important to understand both its strengths and weaknesses. Waterfall works well in certain situations but can cause serious problems in others. This section explains the key benefits and limitations of helping project managers make informed decisions.

Benefits of Waterfall

- **Simple and Clear Structure:** Each phase must be completed before the next begins. There is no confusion about what comes next or when a phase is finished.
- **Easy Progress Tracking:** Each phase has clear deliverables. Project managers can easily see where the project stands by checking which phases are complete.
- **Strong Documentation:** Every phase produces documentation that explains what was decided and why. If team members leave, the knowledge stays with the project.
- **Accurate Estimates**: Requirements are known upfront, so time and cost can be planned reliably from the beginning.
- **Clear Milestones:** Each phase ends with a specific deliverable, making it easy to report progress to stakeholders and management.

Limitations of Waterfall

- **No Going Back:** Once a phase is complete, revisiting it is expensive and difficult. If requirements change after development has started, the project may need to start over.
- **Late Customer Feedback:** Customers see working software only at the end of the project. This means feedback arrives when changes are most costly to make.
- **Assumes Perfect Planning:** Waterfall requires complete and stable requirements upfront. Most software projects cannot provide this because customers often do not know exactly what they want until they see something working.
- **Delayed Value:** No working software is delivered until the final phase. The business gets no return on investment until the very end of the project.
- **Late Risk Discovery:** Technical problems or requirement gaps may not be found until the testing phase, when fixing them is very expensive.

2.3 When to use Waterfall

The Waterfall approach works best in projects where there is clarity, stability, and a clear end goal from the beginning. Since the process follows a fixed sequence, it is most effective when there are little uncertainty and minimal need for changes along the way.

Best Situations for Using Waterfall

Clear and fixed requirements

When the project goals and requirements are fully understood early on, Waterfall allows teams to plan and execute without interruptions.

Low likelihood of change
If changes are rare or discouraged, this approach helps maintain control and keeps the project on schedule.
Well defined scope
Projects with a strict scope and clear boundaries benefit from the structured nature of Waterfall.
Regulated or formal environments
Industries that require strong documentation and approvals, such as construction, healthcare, or government projects, often rely on Waterfall.
Smaller or straightforward projects
When the project is not overly complex, a linear approach is easier to manage and track.

When to Avoid Waterfall

Waterfall may not be suitable when:

- Requirements are unclear or likely to evolve.
- Frequent feedback and iteration are needed.
- The project involves innovation or experimentation.
- Stakeholders need to be involved throughout development.

Chapter 3: Agile & Lean Project Management

3.1 Scrum Framework

Scrum is a lightweight agile framework that helps teams deliver value through short, time-boxed cycles called Sprints. It is built on transparency, inspection, and adaptation (Schwaber & Sutherland, 2020).
Key Idea: Work is done in small, repeatable chunks, and the team continuously improves based on feedback.

Scrum Roles and Responsibilities

Scrum has three core roles:

Role	Who	Key Responsibilities
Product Owner	Usually one person, who is the client	• Manages Product Backlog • Prioritizes work based on value • Accepts or rejects completed work
Scrum Master	One person, part of the development team	• Ensures Scrum process is followed • Removes obstacles for the team • Facilitates meetings and protects team • Has a handle on the whole project
Development Team	Usually 3-9 members. if the team working on the project is bigger, there's multiple Dev teams	• Delivers "Done" work each Sprint • Self-organizes to complete tasks • Owns quality and commitments

Note: In Scrum, there is no Project Manager. These three roles share the responsibilities of a Project Manager.

Scrum Events

Event	Purpose	Timebox
Sprint	Container for all work	1–4 weeks (we use 1 week)
Sprint Planning	Plan what to deliver and how	2 hours per 1-week Sprint
Daily Scrum	Sync on progress, plan next 24 hours	10 - 15 minutes
Sprint Review	Show completed work, get feedback	20-40 minutes
Sprint Retrospective	Reflect and improve process	1-2 hours

Sprint Details and Sprint Planning

Sprint: A fixed timebox where the team creates a "Done" increment of value. Usually one sprint is 1-4 weeks, depending on the size of the project and team.

Sprint Planning answers two questions:

1. **What** can we deliver this Sprint? (Select items from Product Backlog)
2. **How** will we do the work? (Break items into tasks)

Outputs:

- Sprint Goal (why this Sprint matters)
- Sprint Backlog (selected items + plan)

Sprint Review

Held at the end of each Sprint to inspect the “Done” increment and adapt the Product Backlog.

What happens:

- Team presents completed work.
- Product Owner gives feedback.
- Product Owner updates priorities based on feedback.
- Everyone discusses what to do next.

Outcome: Revised Product Backlog for the next Sprint.

Sprint Retrospective

Held after the Sprint Review. The team inspects how they worked and creates a plan for improvement.

Example format: 4 L’s

Category	Question
Liked	What went well?
Learned	What new insights did we gain?
Lacked	What were we missing?
Longed For	What would make our work better?

Process: Each member participates in the discussion and gives input for improvements. The team selects the most important improvement(s) to focus on and makes an actionable plan to improve on those aspects.
Outcome: Actionable improvements for next Sprint.

Scrum Artifacts

Artifact	Description	Owner
Product Backlog	Ordered list of everything needed in the product	Product Owner
Sprint Backlog	Selected items + plan for current Sprint	Development Team
Increment	Sum of completed work that meets Definition of Done	Development Team

Commitments:

- Product Backlog → Product Goal
- Sprint Backlog → Sprint Goal
- Increment → Definition of Done (our quality checklist)

Self-Assessment by Role

For Product Owner:

- Can I prioritize the Product Backlog based on value?
- Do I understand how to accept or reject work?
- Can I communicate the Product Goal clearly?

For Scrum Master:

- Can I facilitate all Scrum events effectively?
- Do I know how to remove team impediments?
- Am I protecting the team from interruptions?
- Do I have a firm grasp of the project schedule and scope?

For Development Team:

- Do I understand our Sprint Goal?
- Can I self-organize to complete tasks?
- Do I know what "Done" means for our team?
- Do I know and understand the processes the team uses?

3.2 User Stories & Scrum Boards

What are User Stories?

A **user story** is a short, simple description of a feature told from the perspective of the person who wishes for the new capability, usually a user or customer of the system. Its purpose is to articulate how a piece of work will deliver value back to the customer.

Standard format:

As a [type of user], I want [an action] so that [benefit/value].

Example (software project):
As a registered user, I want to reset my password so that I can regain access to my account if I forget it.
User stories are intentionally brief. Details are clarified through conversations with the product owner and acceptance criteria. They are **not detailed specifications** – they are placeholders for future discussion.

How User Stories are used in the Sprint

1. **Product Backlog:** All user stories are kept in the product backlog, prioritized by the product owner based on business value, risk, workload, and dependencies.
2. **Sprint Planning:** The team selects a set of user stories from the top of the product backlog that they can commit to completing during the sprint. These stories are moved to the **sprint backlog**.
3. **Sprint Execution:** Each user's story is broken down into smaller, actionable tasks (technical steps). Team members sign up for tasks, update their status, and collaborate to deliver the user story by the sprint end.
4. **Definition of Done:** A story is considered complete only when it meets the team's "Definition of Done" (e.g., code reviewed, tested, documented, deployed to staging).

Tip: As a team member, always ask: *"What value does this story bring?"* and *"What is the acceptance criteria?"* Before starting work.

Introduction to the Scrum Board

The **Scrum board** is a visual tool that displays all the work items in the current sprint and their status. It provides a real-time snapshot of the sprint progress and helps the team coordinate.
Common board columns:

- **To Do:** Stories/tasks have not yet started.
- **In Progress:** Work actively being done.
- **In Review / Test:** Work awaiting peer review, testing, or product owner validation.
- **Done:** Work that meets the Definition of Done.

Boards can be **physical** (sticky notes on a whiteboard) or **digital** (Jira, Trello, Azure Boards). Digital boards are typical for distributed teams.

How to read and use the Scrum Board

Reading the Board

- **Column headers** tell you the status of work.
- **Cards** stand for user stories or tasks. Each card usually holds:
    - Title, description, story points/workload estimate
    - Assignee(s)
    - Acceptance criteria checklist
    - Labels, due dates, attachments (digital boards)

- **Work in Progress (WIP) limits** (optional but recommended): A number next to a column shows the maximum number of items allowed in that column at once. This prevents bottlenecks.

Using the Board Effectively

- **Start your day** by looking at the board during the Daily Scrum. See what moved, what is blocked, and what needs help.
- **Pull work, don't be pushed:** Take a new task from the "To Do" column only when you have capacity.
- **Identify blockers:** If a card stays in "In Progress" for multiple days, it may be blocked. Flag it (e.g., add a red label) and discuss it in the Daily Scrum.

Updating Task Status on the Scrum Board

Keeping the board up-to-date is every team member's responsibility. It ensures transparency and reliable progress tracking.

Step-by-step for digital boards (e.g., Trello, Jira):

1. **When you start a task:**
    a. Move the card from **To Do** → **In Progress**.
    b. Assign yourself as the owner (if not already assigned).
2. **While you work:**
    a. Update checklists, add comments, and attach relevant documents.
    b. If you finish a subtask, mark it as complete.
3. **When you finish a task:**
    a. Move the card to **In Review / Test** (or directly to **Done** if the team workflow allows).
    b. Notify the reviewers/testers via agreed upon method.
4. **When a story is fully completed:**
    a. The reviewer moves the card to **Done**.
    b. Ensure all acceptance criteria are met.

Important: Update the board **at least daily**, preferably at the same time when you change task status. This keeps the Sprint Burndown chart exact and helps the Scrum Master track progress.

Using the Scrum Board to track Sprint progress

The Scrum board is not just a task list – it's a powerful progress-tracking tool.

Sprint Burndown Chart

Most digital Scrum tools automatically generate a **burndown chart** based on board updates. It shows the remaining work (usually story points or hours) over time. If the burndown line is above the ideal line, the team is behind; if below, ahead.

Velocity

After several sprints, the board history helps calculate **velocity** – the average number of stories points the team completes per sprint. Velocity is used to forecast how much work can be done in future sprints.

3.3 Product Backlog & Sprint Backlog

A Product Backlog is a simple list of all the project-related work that needs to be done to build a deliverable product. Simply, Product Backlog is a to-do list for the whole product. This list can include for example new features, bug fixes, improvements and technical work.

Why is a Product Backlog important?

- It helps the team understand what work/tasks are needed to build a product.
- It shows what to do first and what to do later based on the value, risk, and importance.
- It helps the team focus on important work.
- It keeps everyone aligned with project goals.

Creating the Product Backlog

Here is the step-by-step idea for how to create a Product Backlog:
Step 1: Understand the product idea (What we are developing or building)
Step 2: Talk to customers or clients (What they need)
Step 3: List out all the requirements that need to be done.
Step 4: Write items as User Stories
Important Note: The Product Backlog is not fixed. There will be added, removed, or changed tasks or items anytime during the actual implementation stage.

Set Priority of Product Backlog Items

Priority of Product backlog will be decided based on the above-mentioned crucial factors:
1. Value: Which task gives more value to the user or business? (High value = higher priority)
2. Risk: Which task is risky or complex? (High risk tasks are done earlier, so rectifying and cutting risks at earlier stage is possible)
3. Cost / Effort: How much time and effort are needed? (Small but valuable tasks can be done early)
Important Note: Usually, the Product Owner sets priority for the product backlog, while the Scrum Master helps the team to understand and follow it.

Reviewing and refining the Product Backlog

Backlog Refinement means reviewing and improving the product backlog regularly based on the customer review meetings and feedback.

Why is refinement needed?

- To add more details to tasks
- To break big tasks into small tasks
- To remove unnecessary items
- To keep the backlog updated

How is refinement done?

- Team reviews backlog together during sprint meeting.
- Done before Sprint Planning
- Happens regularly, not once.

Aligning the Product Backlog with goals and user requirements

The team must make sure that each backlog item must support project goals and match user needs and requirements.

How to align the backlog?

- Clearly define product goals
- Understand what (features/app specification) users really want.
- Remove items that do not support product goals.
- Update backlog based on feedback of the customer.

Important Note: Every backlog item should help achieve the product goal.

Understanding high level project requirements

What does this step involve?

During the kickoff meeting or initial client/customer discussions, the Product Owner gathers:

- The overall project vision
- Business goals and constraints
- Key features expected by stakeholders
- Success criteria and assumptions

Why is this important?

This step helps create a **clear understanding of the value** the customer expects the product to deliver.

It sets the **foundation for backlog creation** and prevents misalignment later in development.

Breaking Requirements into Smaller User Stories

How does the Product Owner break down requirements?

- Find distinct user goals inside large features.
- Split complex requirements into manageable, testable slices
- Ensure each story delivers value independently.
- Avoid technical tasks or overly detailed instructions at this stage.

Key Principle

“Small enough to complete in a single sprint, valuable enough to deliver on its own.”

This supports incremental delivery and makes estimation easier during Sprint Planning.

Writing clear, small, and actionable User Stories

User Story Template

As a **<type of user>**, I want **<a specific goal>**, so that **<benefit/value>**.

Good user stories are:

- **Concise:** focused on a single objective
- **User-centric:** written from the end-user’s perspective.
- **Independent:** can be planned without depending on other stories
- **Testable:** measurable using clear acceptance criteria

Defining acceptance criteria for each Story

Acceptance criteria describe the conditions under which a story is considered done.

Why is this important?

- Ensures shared understanding between PO and team.
- Sets clear boundaries for scope.
- Makes stories testable and verifiable.

Typical Acceptance Criteria Format

Given **<context>**

When **<action>**

Then **<expected outcome>**

Clear acceptance criteria allow the team to confirm the completion of each story.

Adding User Stories to the Product Backlog

What does the Product Owner do next?

- Add all newly written user stories to the Product Backlog
- Prioritize based on business value, dependency, and risk.
- Continuously update and refine the backlog as new insights appear.
- Ensure the backlog is transparent and understood by the entire team.

Purpose:

A well-structured Product Backlog enables effective sprint planning, refined prioritization, and predictable delivery of value.

3.4 Other Agile Frameworks

While Scrum is one of the most widely adopted agile methodologies, several other frameworks exist that may be better suited for specific project types, team structures, or organizational scales. This section provides a brief overview of three prominent alternative agile frameworks: **Extreme Programming (XP)**, **Scaled Agile Framework (SAFe)**, and **Feature-Driven Development (FDD)**. Understanding these alternatives enables Scrum Masters and teams to make informed decisions when Scrum may not be the best fit.

Extreme Programming (XP)

Extreme Programming is an agile framework that emphasizes **technical excellence**, **customer satisfaction**, and **rapid feedback loops**. It is particularly focused on improving software quality and responsiveness to changing customer requirements (Beck, 2000).

Core Practices:

- **Pair Programming:** Two developers work together at one workstation, continuously reviewing code.
- **Test-Driven Development (TDD):** Developers write automated tests before writing the corresponding code.
- **Continuous Integration:** Code changes are integrated and tested multiple times per day.
- **Small, Frequent Releases:** Functional software is delivered in noticeably short cycles, typically every 1–2 weeks.

When to Consider XP:

- Projects where **software quality and reliability** are paramount.
- Teams that need very **rapid feedback** from customers or stakeholders.
- Environments that encourage **close collaboration** and collective code ownership.

Scaled Agile Framework (SAFe)

The Scaled Agile Framework is designed for **large enterprises** that need to coordinate dozens or hundreds of agile teams while keeping alignment, transparency, and program-level execution (Leffingwell, 2007).

Key Components:

- **Agile Release Trains (ARTs):** Long-lived teams of agile teams that plan, commit, and execute together.
- **Program Increment (PI):** A fixed timebox, typically 8–12 weeks, during which an ART delivers incremental value.
- **Essential Roles:** Release Train Engineer (RTE), Product Management, System Architect.
- **Four-Level Structure:** Team, Program, Large Solution, and Portfolio levels provide clarity at different scales of operation.

When to Consider SAFe:

- **Large organizations** (typically 50+ practitioners) require coordination across multiple teams.
- Projects needing **significant alignment** between business strategy, portfolio management, and team execution.
- Environments with **complex compliance, governance, or regulatory requirements**.

Feature-Driven Development (FDD)

Feature-Driven Development is a model-centric, iterative agile framework that organizes work around the **delivery of client-valued features** (Palmer & Felsing, 2002).

Five Core Activities:

1. **Develop an Overall Model:** Create a high-level object model for the domain.
2. **Build a Feature List:** Show and list all feature to be developed.
3. **Plan by Feature:** Prioritize features and assign them to Chief Programmers.
4. **Design by Feature:** Create detailed design packages for each feature.
5. **Build by Feature:** Implement, test, and integrate each feature.

When to Consider FDD:

- **Large-scale, long-duration projects** with well-defined domains and features.
- Teams that receive help from a **strong emphasis on upfront modelling and design**.
- Organizations with **experienced domain experts and developers** who can drive feature development.

Framework Comparison and Selection Guidance

Aspect	Scrum	XP	SAFe	FDD
Primary focus	Process, roles, and timeboxing	Engineering practices and quality	Scaling and enterprise alignment	Feature delivery and domain modeling
Team Size	3–9 people (per team)	Small teams	50–500+ people	Medium to large teams
Iteration Length	2–4 weeks (Sprint)	1-2 weeks	8–12 weeks (Program Increment)	Feature-based (weeks to months)
Best Suited For	General-purpose agile projects	High-quality, adaptive code bases	Large, multi-team organizations	Large projects with clear features

3.5 Waterfall vs Agile – Choosing the right approach

One of the most common questions project managers faces is: **"Should we use Waterfall or Agile?"** There is no universal answer—the right choice depends on your project's characteristics, constraints, and context (Boehm & Turner, 2003). This section provides practical guidance to help you make an informed decision.
Both methodologies have proven successful in different scenarios. The key is understanding their core differences and matching them with your project's needs.

Decision Framework: Key Factors to consider

Use these six factors to evaluate which approach fits your project:

Factor	Waterfall Suits	Agile Suits
Requirements Clarity	Requirements are well-understood, stable, and unlikely to change	Requirements are uncertain, evolving, or expected to change
Customer Involvement	Customers are available primarily at milestones (requirements and delivery)	Customer available for continuous feedback throughout
Project Complexity	Simple to moderately complex projects with clear dependencies	Complex projects where unknowns exist and solutions emerge over time
Team Size & Location	Larger teams, potentially distributed, with clear handoffs	Small, co-located, cross-functional teams (though scaled Agile exists)

Time to Market	When full feature set is required before launch	When early delivery of core value is needed with iterative enhancements
Risk Profile	Low technical risk; requirements-driven risk	High technical risk: welcomes change and adaptation

Comparison Table: Waterfall vs Agile

Aspect	Waterfall	Agile
Philosophy	Plan-driven; predictable and repeatable	Value-driven; adaptive and responsive
Approach	Sequential phases	Iterative increments
Requirements	Defined upfront and fixed	Emergent and continuously refined
Customer Feedback	End of project only	Continuous throughout
Delivery	Single final delivery	Frequent small deliveries (every 1-4 weeks)
Team Structure	Specialized roles with handoffs	Cross-functional, self-organizing teams
Documentation	Comprehensive documentation	Just enough documentation
Change Management	Formal change control process	Changes welcomed and absorbed in backlog
Success Metric	On-time, on-budget delivery	Customer value delivered
Best For	Regulated industries, fixed-price contracts, physical products	Software, innovative products, dynamic environments

Hybrid Approaches

Sometimes the best solution combines elements of both. Common hybrid approaches include:

Waterfall with Agile Phases

Use Waterfall for high-level planning and requirements, then execute development in Agile sprints. Useful when:

- Fixed budget/contract required upfront.
- Development work benefits from iteration
- Regulatory requirements demand certain documentation.

Agile with Waterfall Elements

Use Agile for development but incorporate Waterfall practices like:

- Phase gates for major releases
- Comprehensive documentation for compliance
- Fixed milestones for stakeholder reporting

Example: Wagile or Water-Scrum-Fall

Common in organizations transitioning to Agile:

- Requirements gathered upfront (Waterfall)
- Development in sprints (Scrum)
- Final testing and deployment as separate phase (Waterfall)

Real-World Examples

When Waterfall Succeeded

Project: Bridge Construction

- Requirements fixed and regulated.
- Changes extremely costly
- Sequential phases essential
- Customer involvement at milestones only
- **Verdict:** Waterfall was clearly the right choice

When Agile Succeeded

Project: Mobile Banking App Startup

- Requirements evolved with user testing
- Needed early market entry with core features.
- Team of 6 co-located developers.
- Weekly releases to gather feedback.
- **Verdict:** Agile enabled rapid iteration and market fit

When Hybrid Worked

Project: Government Healthcare Portal

- Fixed budget and regulatory requirements (Waterfall)
- User experience needed iterative improvement (Agile)

- Solution: Waterfall for compliance documentation and milestones; Agile for UI/UX development
- **Verdict:** Hybrid satisfied both constraints

Key Takeaways

1. **No single methodology fits all projects** – context matters.
2. **Assess your project** using the six-factor framework.
3. **Use the decision matrix** to quantify your analysis.
4. **Consider hybrid approaches** when pure models don't fit.
5. **Re-evaluate over time** – what works now may change as your project evolves.

Decision guides

Decision Matrix

Use this simple scoring matrix to evaluate your project. For each factor, assign a score:

- **+1** = Favors Agile
- **0** = Neutral / Both possible
- **-1** = Favors Waterfall

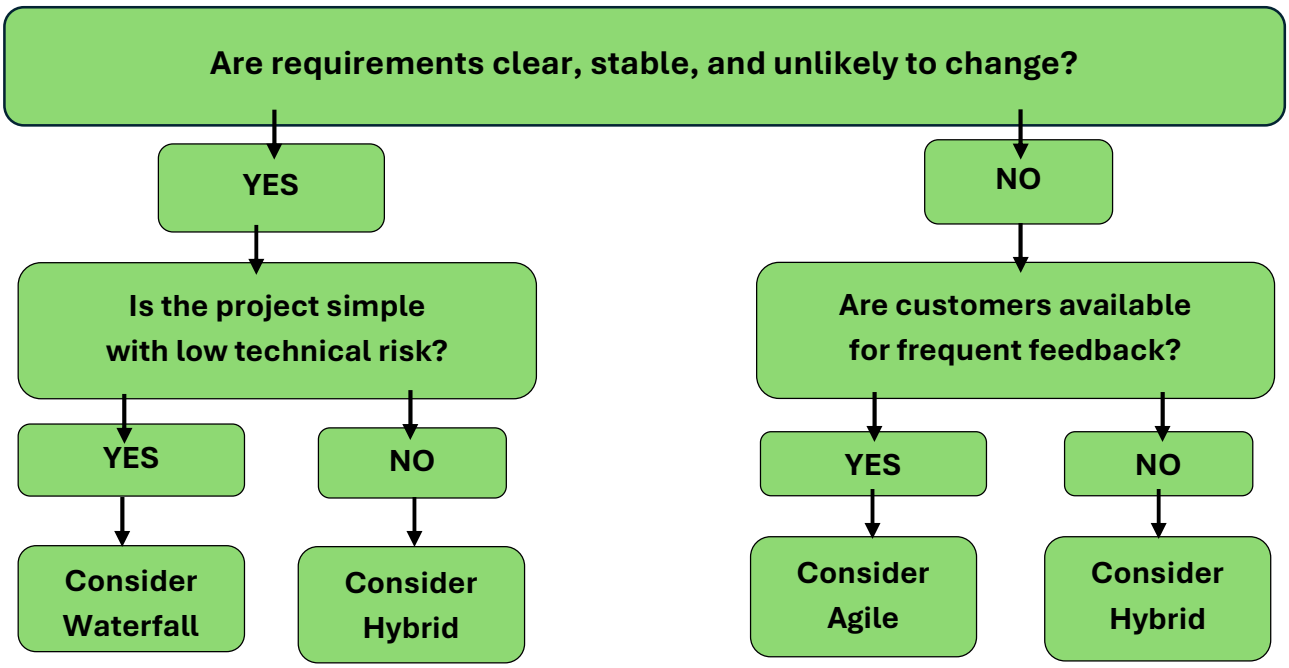
Factor	Score	Notes
Requirements are unclear or likely to change	+1	
Requirements are stable and well-defined	-1	
Customer is available for frequent feedback	+1	
Customer wants fixed scope and price	-1	
Project is time-critical with MVP needed early	+1	
Project must deliver all features at once	-1	
Team is cross-functional and co-located	+1	
Team is specialized and geographically distributed	-1	
High technical uncertainty or innovation needed	+1	
Low technical risk, well-understood technology	-1	
TOTAL SCORE		

Interpreting Your Score:

Score Range	Recommendation
+6 to +11	Strongly Favors **Agile**

+1 to +5	Lean toward **Agile** – consider hybrid approaches
0	Either could work – consider other factors
-1 to -5	Lean toward **Waterfall** – consider hybrid
-6 to -11	Strongly Favors **Waterfall**

Decision Flowchart



Chapter 4: WBS & GANTT Chart

4.1 Work Breakdown Structure (WBS)

What is a Work Breakdown Structure?

A Work Breakdown Structure **(WBS)** is a hierarchical decomposition of the total scope of work required to complete a project. It breaks a large, complex project into smaller, more manageable pieces called work packages.

Think of **WBS** as a project map. Instead of seeing one overwhelming goal ("Build a house" or "Create a guidebook"), the WBS shows all the small tasks that together make up the whole project.

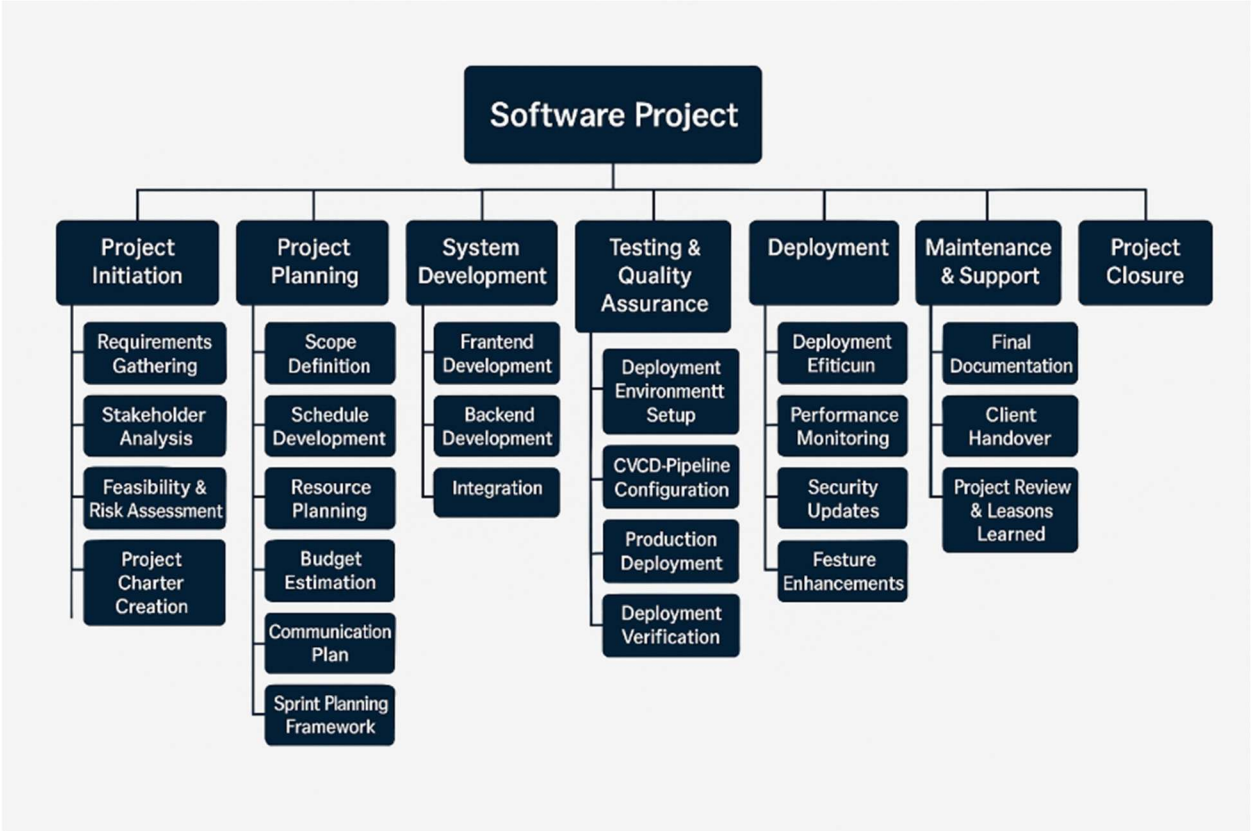
4.2 How to create a WBS

A **Work Breakdown Structure (WBS)** is a hierarchical decomposition of the total work required to deliver a project.

For a software project, the WBS breaks down complex activities into manageable components, helping the Scrum Master, Product Owner, and development team maintain alignment and clarity.

Below is a **practical, realistic, and easy-to-apply WBS** for a typical software development project.

Example of Software Project WBS Diagram



Explanation of Each WBS Branch:

Project Initiation

This phase establishes the foundation of the project. It involves gathering initial requirements, identifying stakeholders, analyzing risks, and forming the project charter. These steps define the project's purpose and boundaries.

Project Planning

Planning outlines how the project will be executed. This includes defining the project scope, creating timelines, estimating budget and resources, and preparing communication and sprint plans. Planning ensures clarity before development begins.

System Design

This phase translates requirements into technical models. UI/UX designers create wireframes; architects design the system structure, and technical teams prepare database designs and API specifications. This ensures a clear blueprint for development.

Software Development

The core execution phase where code is created. Frontend developers build the visual interface, while backend developers implement logic, data handling, and API functionalities. Integration ensures both sides work seamlessly together.

Testing & Quality Assurance

Testing validates the quality and functionality of the software. The team conducts unit, integration, system, and user tests to identify and fix defects before release. This guarantees reliability and user satisfaction.

Deployment

The working system is released to the production environment. Activities include configuring servers, setting up CI/CD pipelines, performing the deployment, and verifying the system in its live environment.

Maintenance & Support

After deployment, the team continues monitoring performance, resolving issues, and updating the system with improvements or security patches. This ensures long-term stability and user adoption.

Project Closure

The final phase includes delivering all documentation, handing over the system to the client, and reviewing lessons learned. It marks the formal completion of the project.

4.3 The GANTT Chart

GANTT is the most used tool to help schedule and track any project's tasks and timeline visually. It's also used to determine and show the project's task dependencies and durations, critical path, and milestones. These are presented as a horizontal bar chart with arrows presenting the task dependencies and critical path.

GANTT chart can't be created without a WBS; all the tasks and milestones are taken from there.

Key components

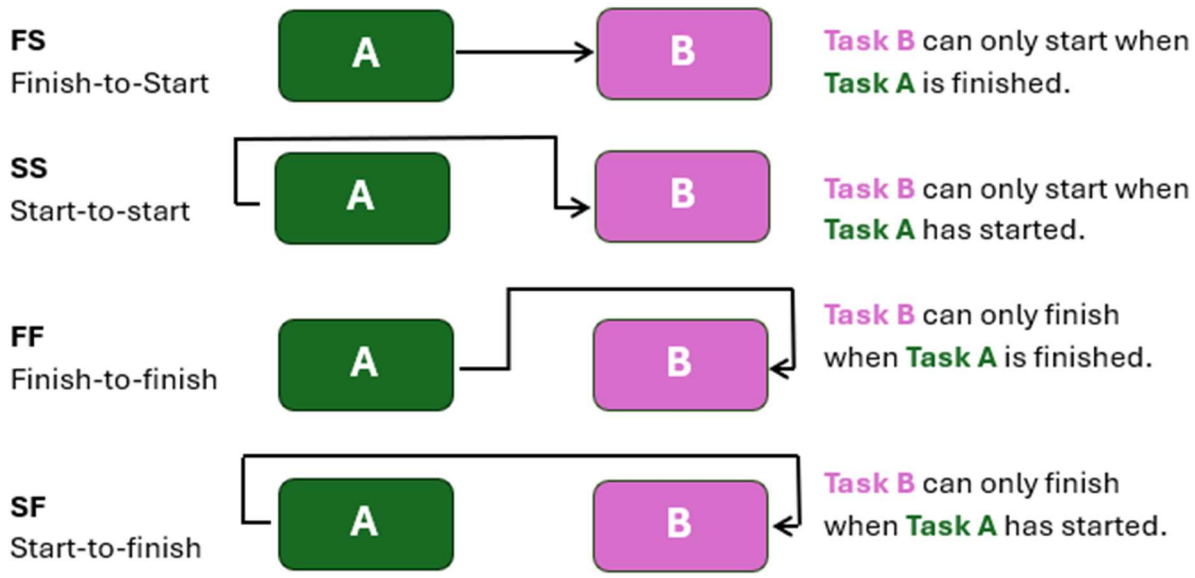
1. **Task bars:** These horizontal bars are a visual representation of all the different tasks within the project. The length of the bar tells the duration of the task, and the position shows the start and end dates.
2. **Task dependencies**: These are shown with arrows, and they show the different relations between tasks; they indicate when a task can start or finish relative to others. There are 4 different types of dependencies (shown on the next page)
3. **Critical Path:** The sequence of tasks that directly impact the duration of the whole project. Basically, it's the longest sequence of dependent tasks and any delay in one of the tasks will directly impact the project's completion.
4. **Milestones:** These are usually shown as a diamond shape in the chart. They don't have a duration, because they mark a significant activity in the project, for example approving a new UI design for a website.
5. **Task allocation**: This tells who are assigned to each task, which helps to give a clear overview of resource distribution through the project. This helps with balancing the workload between people and preventing possible bottlenecks.

Additional features:

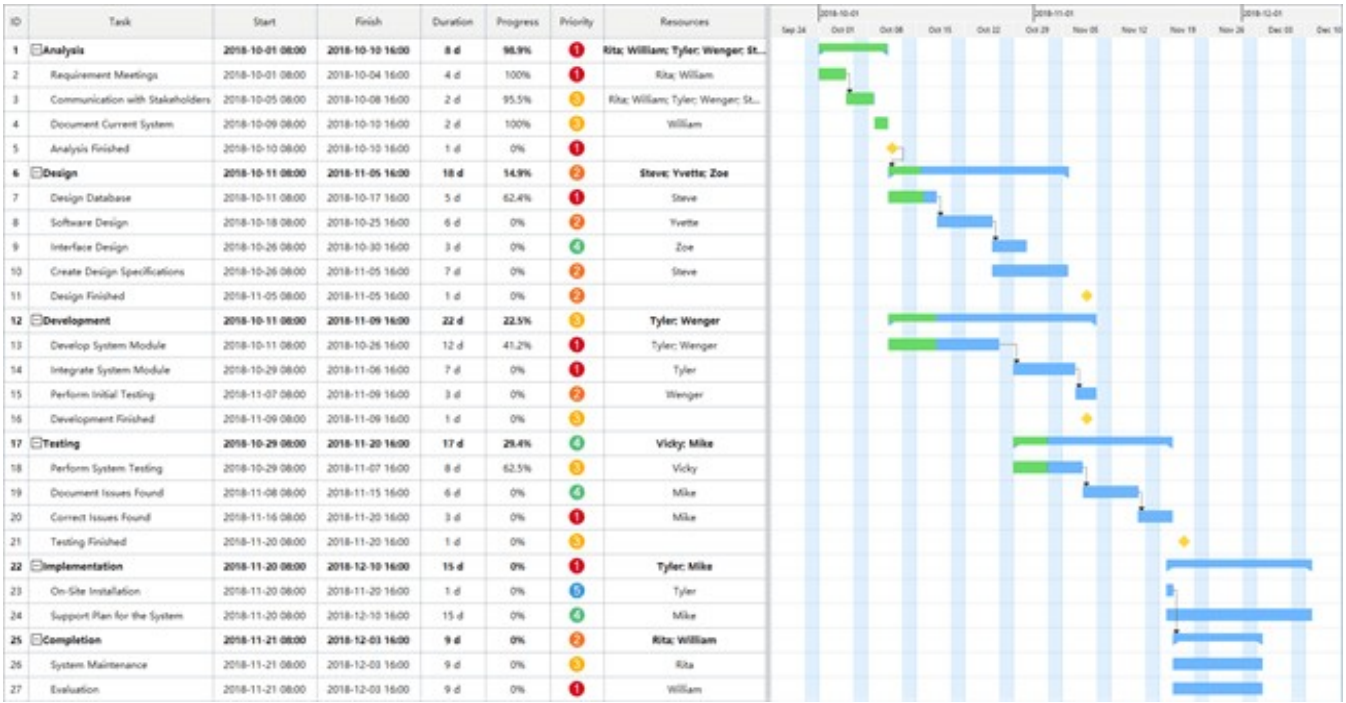
- **Task priority:** Shows the task's priority in the scale the company/group uses, for example: LOW/MEDIUM/HIGH or number scale.

- **Task duration in days**: Task duration is already shown by the horizontal bars, but they can also be shown as a number and the abbreviation of “day”: 5d.
- **Progress in percentages:** Especially in bigger projects this is helpful to keep track of progress and notice possible problems, if something isn’t progressing.

Task dependencies:



Example of a GANTT chart in a software project:



ID	Task	Start	Finish	Duration	Progress	Priority	Resources
1	**Analysis**	**2018-10-01 08:00**	**2018-10-10 16:00**	**8 d**	**98.9%**	1	**Rita; William; Tyler; Wenger; St...**
2	Requirement Meetings	2018-10-01 08:00	2018-10-04 16:00	4 d	100%	1	Rita; William
3	Communication with Stakeholders	2018-10-05 08:00	2018-10-08 16:00	2 d	95.5%	3	Rita; William; Tyler; Wenger; St...
4	Document Current System	2018-10-09 08:00	2018-10-10 16:00	2 d	100%	3	William
5	Analysis Finished	2018-10-10 08:00	2018-10-10 16:00	1 d	0%	1	
6	**Design**	**2018-10-11 08:00**	**2018-11-05 16:00**	**18 d**	**14.9%**	2	**Steve; Yvette; Zoe**
7	Design Database	2018-10-11 08:00	2018-10-17 16:00	5 d	62.4%	1	Steve
8	Software Design	2018-10-18 08:00	2018-10-25 16:00	6 d	0%	2	Yvette
9	Interface Design	2018-10-26 08:00	2018-10-30 16:00	3 d	0%	4	Zoe
10	Create Design Specifications	2018-10-26 08:00	2018-11-05 16:00	7 d	0%	2	Steve
11	Design Finished	2018-11-05 08:00	2018-11-05 16:00	1 d	0%	2	
12	**Development**	**2018-10-11 08:00**	**2018-11-09 16:00**	**22 d**	**22.5%**	3	**Tyler; Wenger**
13	Develop System Module	2018-10-11 08:00	2018-10-26 16:00	12 d	41.2%	1	Tyler; Wenger
14	Integrate System Module	2018-10-29 08:00	2018-11-06 16:00	7 d	0%	1	Tyler
15	Perform Initial Testing	2018-11-07 08:00	2018-11-09 16:00	3 d	0%	2	Wenger
16	Development Finished	2018-11-09 08:00	2018-11-09 16:00	1 d	0%	3	
17	**Testing**	**2018-10-29 08:00**	**2018-11-20 16:00**	**17 d**	**29.4%**	4	**Vicky; Mike**
18	Perform System Testing	2018-10-29 08:00	2018-11-07 16:00	8 d	62.5%	3	Vicky
19	Document Issues Found	2018-11-08 08:00	2018-11-15 16:00	6 d	0%	4	Mike
20	Correct Issues Found	2018-11-16 08:00	2018-11-20 16:00	3 d	0%	1	Mike
21	Testing Finished	2018-11-20 08:00	2018-11-20 16:00	1 d	0%	3	
22	**Implementation**	**2018-11-20 08:00**	**2018-12-10 16:00**	**15 d**	**0%**	1	**Tyler; Mike**
23	On-Site Installation	2018-11-20 08:00	2018-11-20 16:00	1 d	0%	5	Tyler
24	Support Plan for the System	2018-11-20 08:00	2018-12-10 16:00	15 d	0%	4	Mike
25	**Completion**	**2018-11-21 08:00**	**2018-12-03 16:00**	**9 d**	**0%**	2	**Rita; William**
26	System Maintenance	2018-11-21 08:00	2018-12-03 16:00	9 d	0%	3	Rita
27	Evaluation	2018-11-21 08:00	2018-12-03 16:00	9 d	0%	1	William

Chapter 5: Scope Management

5.1 Defining Project Scope

Project scope means clearly defining what the project will include and what it will not include. It sets clear boundaries for the project. This will help the Product Owner, Scrum Master, and development team understand exactly what they need to create and deliver.
A well-defined scope makes sure everyone is working toward the same goal and avoids confusion during the actual implementation phase of the project.

What should we include in scope?

The scope should include only the actual work, features, and final deliverables that the project is expected to produce based on the requirements of PO or customer.
For example, in a Project Management Guidebook project, the scope can include:

- Complete content of the guidebook (must cover all the user stories)
- Introduction to project management
- Different project management methods like Waterfall and Agile
- Important processes such as: Work Breakdown Structure (WBS), Gantt charts, Risk management, and Quality management
- All chapters and their detailed sections
- Clear examples and explanations for better understanding (user-perspective)
- Templates and references that can be used in real projects.
- Explanation of project management tools such as Trello, Microsoft planner, and Jira

In short, the scope should include everything required to successfully complete and deliver the guidebook.

What should we exclude from scope?

The scope should exclude anything that is **not part of the final deliverable** or does not directly contribute or be relevant to the project output.
For example, in the same guidebook project, the following can be excluded:

- Developing actual software or tools
- Providing advanced training or certification programs
- Publishing, marketing, or selling the guidebook.

Also, exclude things that are part of internal project work, such as:

- Team-related issues (like communication problems or lack of participation)
- Internal activities (meetings, discussions, planning sessions)

If it is not something the project will deliver to the user or customer, it should not be included in the scope.

Why is Scope important?

Having a clear project scope is very important for project success because:

- It helps everyone understand their roles and responsibilities.
- It reduces confusion about what needs to be done.
- It prevents scope creep (adding extra work that was not planned)
- It helps manage time and complete the project within deadlines.
- It keeps the project within budget and limits.
- It reduces risks by avoiding unnecessary work.

5.2 Preventing Scope Creep

Scope creep happens when new tasks or requirements are added to a project without proper approval. It often starts small: "Can you just add one. more thing?", but these small additions accumulate and cause significant problems over time.

Why does it matter?

Problem	Effect on Our Project
Missed deadlines	Extra work delays our 8-sprint schedule
Team burnout	Too much work overloads team members
Lower quality	Rushing leads to mistakes in the guidebook
Confusion	Original goals become unclear

How to prevent Scope Creep

Technique	How We Use It
Change control process	Any new request goes through our Change Management Plan
Trade-off rule	Adding something means removing something else
Regular backlog review	We prioritize weekly to focus on what matters

What to do when scope creep happens

1. **Spot it** – Notice when work keeps growing beyond the original task.
2. **Raise it** – Bring it up in daily Scrum or team chat.
3. **Apply the rule** – New work needs a change request (per our Change Management Plan)
4. **Ask the trade-off** – "What do we remove to fit this in?"
5. **Communicate** – Let stakeholders know the impact.

Scrum Master's Role

Action	Purpose
Say "no" or "not now"	Protect team from overload
Remind team of change process	Keep work under control
Ask "what can we remove?"	Make trade-offs visible
Flag warning signs early	Prevent problems before they grow

Self-Check

- Can I spot scope creep when it starts?
- Do I know what to do when someone asks for "just one more thing"?
- Can I explain to my team why these things matter?

5.3 Key processes of scope management

Managing scope is an ongoing effort throughout the project. It is about setting clear boundaries, keeping the work organized, and making sure nothing unnecessary slips in.

Scope Planning

At this stage, the team decides how scope will be handled. This includes setting rules for changes and agreeing on how to manage the work. It gives structure to everything that follows.

Scope Definition

This is where the project's scope is clearly written out. It defines what the project will deliver and what it will not be. A clear definition helps avoid confusion and keeps everyone aligned.

Work Breakdown Structure (WBS)

The project is broken into smaller, manageable parts. This makes it easier to assign tasks, track progress, and understand what needs to be done.

Scope Validation

Here, the team checks if the completed work matches the original scope. It involves reviewing deliverables and getting approval before moving forward.

Scope Control

This process handles any changes to the scope. New requests are reviewed carefully, and only approved changes are added, so the project stays focused.

5.4 How to make a scope management plan

A Scope Management Plan is a key part of project planning. It defines how the project scope will be identified, documented, monitored, and controlled throughout the project lifecycle. In simple terms, it acts as a guide that keeps the project focused on what was agreed from the start. A well-written scope plan reduces confusion, prevents unnecessary work, and ensures that all team members understand what is included in the project and what is not.

Define the Project Purpose

Every scope management plan starts with a clear understanding of why the project exists. This section briefly describes the project goal and the expected outcome.

A clear purpose helps set direction and ensures that all scope decisions support the same objective.

Identify Project Deliverables

Deliverables are the actual outputs the project will produce. These should be clearly listed so there is no ambiguity about what the team is responsible for completing. Deliverables can include documents, systems, reports, or in this case, chapters and sections of the guidebook. A clear list of deliverables helps the team stay focused on what must be completed.

Define Scope Boundaries

One of the most important parts of scope planning is clearly stating what is included in the project and what is not. This step prevents misunderstandings and reduces the risk of extra work being added later without control.

In practice, this means:

- Clearly list what part of the project is.
- Clearly stating what is outside the project.
- Making sure both sides are agreed by all stakeholders.

Establish Scope Management Approach

This section explains how scope will be handled during the project. It defines the overall method the team will follow to stay in control of the work.

It usually includes:

- How scope will be tracked
- How updates will be recorded
- How progress will be measured against the original plan

This ensures that scope is not just defined once but actively managed throughout the project.

Define Change Control Process

Changes are common in most projects, but they need to be managed properly.

This part explains how new requests or changes to scope will be handled. No change should be added directly without review.

A structured change process usually includes:

- Submitting a change request
- Reviewing the impact on time, cost, and effort
- Approving or rejecting the change
- Updating the scope if approved

This helps prevent uncontrolled additions that can affect project delivery.

Set Scope Validation Process

Scope validation ensures that completed work matches what was originally agreed upon. This step involves reviewing deliverables and confirming that they meet the defined requirements before moving forward.

It usually includes:

- Review sessions with stakeholders.
- Formal acceptance of completed work
- Feedback and correction if needed.

This ensures the project is delivering the right output, not just completing tasks.

Assigning Roles and Responsibilities

A scope management plan should clearly define who is responsible for what.

Typical roles include:

- **Product Owner**: Approves scope and prioritizes requirements.
- **Project Manager or Scrum Master**: Ensures scope is followed and controlled.
- **Development Team**: Delivers work according to defined scope.

Clear roles reduce confusion and improve accountability within the team.

Chapter 6: Risk Management

6.1 Identifying project risks

Risk management involves identifying, assessing, and addressing potential issues that could affect a software project's success. It is essential because it helps teams anticipate problems early, allowing them to prevent delays, control costs, maintain quality, and meet customer expectations. By identifying risks in advance, project managers and Scrum Masters can take proactive action instead of reacting to problems later.

Three Common Software Project Risks

1. Scope Creep
Uncontrolled expansion of features or requirements, often leading to delays and increased costs.

2. Technical Risks
Issues such as unproven technology, integration difficulties, performance failures, or architectural weaknesses.

3. Resource & Scheduling Risks
Lack of developers, unexpected team turnover, inaccurate estimates, or deadline pressure causing rushed work.

6.2 Risk assessment & analysis

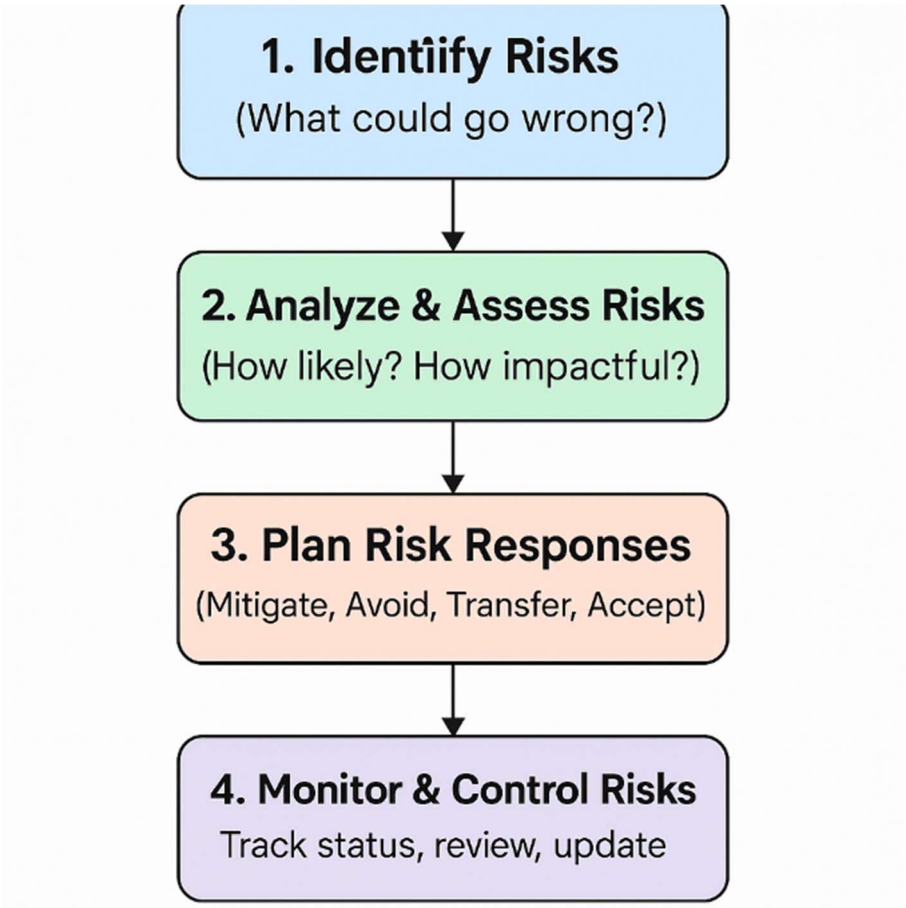
Types of Project Risks

Risk Type	Definition	Simple Example
Technical Risks	Risks that arise from the technology, architecture, tools, or technical solutions used in the project. They typically occur when the team is working with complex, new, or unstable technology.	The team chooses a new JavaScript framework that later turns out to be incompatible with the backend API.
Schedule Risks	Risks that impact the project timeline, causing delays. These occur when tasks take longer than estimated or when dependencies block progress.	A key feature depends on a third-party API, but the vendor delays their release by 2 weeks, pushing the sprint timeline.
Budget Risks	Risks that cause the project to exceed the planned budget, often due to underestimation, unexpected work, or resource costs.	Additional security requirements are introduced mid-project, requiring extra development hours that weren't budgeted.
Team Risks	Risks related to people—availability, skills, communication, or team morale. These often affect productivity and delivery speed.	A senior developer leaves the team during the critical development phase, slowing progress.

Risk Management Process

A simple, systematic process helps project teams identify risks early, analyze their impact, and take meaningful action.
Below is a clear 4-step risk management flow suitable for any project environment.

4-Step Risk Management Flowchart



One-sentence explanation per step

1. Identify Risks
List all potential events that could affect the project's objectives, gathered through brainstorming, past data, and expert input.
2. Analyze & Assess Risks
Evaluate each risk by estimating its probability and impact to determine its priority.
3. Plan Risk Responses
Choose strategies—mitigation, avoidance, transfer, or acceptance—to address high-priority risks effectively.
4. Monitor & Control Risks
Continuously track risks, review status, and update plans as new risks appear or existing risks change.

Complete Example Walkthrough

Below is a realistic example showing the full 4-step process in action.

Project Scenario

A team is developing a new customer portal for a client with a fixed deadline.

Step 1: Identify Risks

The team identifies the following risk:

"The external authentication provider may delay delivering the integration API."

This risk is logged in the risk register.

Step 2: Analyze & Assess the Risk

The team evaluates impact and probability:

- **Probability:** High (vendor has a history of delays)
- **Impact:** High (authentication is mandatory for user login)
- **Risk Score:** High priority!

The risk is marked **RED** in the risk matrix.

Step 3: Plan Risk Responses

The project manager defines the response strategy:

- **Mitigation Strategy:** Begin development with mock API endpoints so work can proceed even if the vendor is late.
- **Contingency Plan:** If the delay exceeds two weeks, switch to an alternative authentication provider.
- **Preventive Action:** Schedule weekly sync calls with the vendor.

Step 4: Monitor & Control

During project execution:

- The PM checks vendor progress during weekly meetings.
- Risk status is updated in every sprint review.
- After 10 days, the vendor confirms on-time delivery: risk severity is downgraded.
- The mock API allows the team to stay on schedule.

Outcome:

The mitigation plan prevents delays, the system is delivered on time, and the team avoids a major project setback.

Summary

Step	Meaning	Outcome from Example
1. Identify	Capture risks early	Vendor delay identified
2. Analyze	Assess probability & impact	Marked high-priority risk
3. Plan Responses	Define mitigation & contingency plans	Mock API + backup provider
4. Monitor & Control	Track and update risk status	Risk reduced; schedule protected

6.3 Risk register & mitigation

A risk register is the project team's running log of identified risks and the actions taken against them. It turns risk management from a one-time exercise into a living document the team revisits at every status meeting.

A typical risk register has one row per identified risk and the following columns:

Field	Description
Risk ID	Unique identifier (e.g., R-001)
Description	Short statement of the risk event and its potential cause
Category	Type of risk (technical, schedule, resource, external, etc.)
Probability	Likelihood of occurrence (Low / Medium / High, or 1–5)
Impact	Severity if it occurs (Low / Medium / High, or 1–5)
Risk Score	Probability × Impact — used to prioritize
Owner	Team member responsible for monitoring and acting
Mitigation Plan	Action(s) to reduce probability or impact
Contingency Plan	Fallback action if the risk materializes
Status	Open / Mitigated / Closed / Occurred

Example risk register

ID	Description	Prob.	Impact	Score	Owner	Mitigation	Status
R-001	Key developer leaves mid-sprint	M	H	9	PM	Cross-train two team members; document critical work	Open
R-002	Third-party API delay	M	M	6	Tech Lead	Schedule weekly sync; identify backup provider	Open
R-003	Scope creep from late stakeholder requests	H	M	9	PM	Enforce change control process; weekly stakeholder review	Mitigated
R-004	Performance issues at launch	L	H	6	QA Lead	Load testing in sprint 8 and 12	Open

Chapter 7: Change Management

7.1 What is Change Management

Change management is simply about handling changes in a project in a clear and controlled way. Instead of adding things randomly, every new request is reviewed before it becomes part of the work.

Changes are normal in any project. New ideas come up or requirements shift. Without a proper process, this can lead to confusion, delays, and extra work. Change management helps keep things organized and on track.

Why it matters

It helps the team:

- Stay focused on the original goals.
- Avoid unnecessary work.
- Understand the impact of changes.
- Keep the project within time and limits.

How it works

In simple terms:

1. A change is suggested.
2. The impact is reviewed.
3. It is approved or rejected.
4. If approved, it is added properly.

7.2 Change Management Plan

A change management plan is a document that defines how changes to the project are requested, evaluated, approved, and implemented. It ensures that every change goes through a consistent process rather than being added randomly.

For our project, this plan prevents scope creep and protects our 8-sprint timeline.

Purpose and Objectives

Objective	Why It Matters
Control scope	Prevents uncontrolled additions to the project
Clear process	Everyone knows what to do when a change is requested
Fair decisions	Changes are evaluated based on value and impact, not urgency or pressure
Traceability	Every approved change is documented and tracked

Key Components of a Change Management Plan

A basic change management plan includes these parts:

1. **Change Request Process:** How changes are submitted, evaluated, and approved
2. **Roles and Responsibilities:** Who does what (Product Owner, Scrum Master, Team)
3. **Approval Workflow:** The step-by-step path from request to decision
4. **Documentation:** Where and how approved changes are recorded

How to Create a Change Management Plan

Follow these steps to write your own plan:

Step 1: Define the Process

Decide how changes are submitted. Create a simple form. Set a timeline for responses.

*Example:*Change requests are submitted by Product Owner during Sprint Review or via email.

Step 2: Assign Roles

Role	Responsibility
Product Owner	Requests changes, evaluates value, makes final decision
Scrum Master	Creates backlog cards, facilitates impact analysis, ensures process is followed
Team	Provides effort estimates, implements approved changes

Step 3: Set Approval Criteria

Define how decisions are made. What questions to ask.

Example from our project: Is there value? What is the effort? What trade-off is required?

Step 4: Document Everything

Create a simple change request form. Decide where to store approved changes (Trello, email, shared drive).

How to Follow the Change Management Plan (For Team Members)

As a team member, here is how you use the plan:

When the Product Owner requests a change:

- Product Owner submits the change request during Sprint Review or via email.
- Scrum Master adds it to the backlog with a change request card.
- Team evaluates effort and impact in the next Sprint Planning
- If approved, the change is scheduled like any other task.

If you want to suggest a change:

- Discuss with Product Owner first
- If Product Owner agrees, they will submit a formal request.
- You do not submit changes directly.

When you are asked to evaluate a change:

- Estimate the effort required.
- Identify any dependencies or risks.
- Provide your estimate to the Product Owner

When a change is approved:

- Scrum Master adds it to the Product Backlog
- The change is prioritized by Product Owner
- Team schedules it for the next Sprint Planning

When you notice scope creep:

- Raise it in daily Scrum.
- Ask: "Has this been approved through the change process?"
- Remind the team to follow the plan.

7.3 Change Request Process

A change request is a formal way to propose updates to project work — adding content, shifting deadlines, or modifying deliverables — through a controlled process. Because the team works in fixed-deadline sprints, every change must be recorded, assessed, and approved before it enters the backlog.

Why a change request is needed

Reason	What it ensures
Records every change	Full audit trail of decisions
Avoids scope creep	Work stays within agreed boundaries
Prevents last-minute surprises	Risks surfaced early, not at delivery
Communicates clearly	Everyone knows what changed and why
Forces a deliberate decision	Team accepts or rejects with full context

The 8-step change request process

Step	Action	Owner
1. Identify the change	Spot that something needs updating or improvement.	Any team member
2. Inform Scrum Master	Contact the Scrum Master, usually by email.	Requester
3. Fill the form	Complete the change request form: what, why, when, priority.	Requester
4. Submit the form	Submit within the agreed window (e.g., 24 hours).	Requester
5. Review	Check completeness; ask clarifying questions if needed.	Scrum Master
6. Impact analysis	Assess effect on timeline, workload, scope, quality.	Team
7. Decision	Small → Scrum Master; Medium → SM + PO; Big → whole team.	Per change size
8. Implementation	If approved: add to backlog, plan into next sprint, document.	Team

Example: a team member proposes adding a new chapter topic. They inform the Scrum Master, fill the form, the team assesses impact, and — if approved — it enters the next sprint.

Evaluating the impact of a change

The Scrum Master assesses how a proposed change affects the project's three core constraints — scope, time, and quality. Adjusting one almost always forces a trade-off in the others.



When this changes...	Likely trade-off
Scope increases (more features)	**Needs more time, or quality drops**
Time is reduced (tighter deadline)	**Reduce scope, or quality drops**
Quality bar raised (more testing/standards)	**Needs more time, or scope reduced**

How a change can affect risks

Risk type	Trigger
Delay	Work increases or rework needed
Budget / resource	Extra effort or external cost
Quality	Work rushed to absorb the change
Technical	New features add complexity
Team overload	Too many concurrent changes

Change log

A change log records every requested change and its outcome. It maintains the history of decisions, tracks approvals and rejections, supports accountability, and provides an easy reference for future audits.

Required fields

Field	What to record
Change ID	Unique reference (e.g., CR-01)
Date	When the change was requested
Description	What is being changed and why
Requested by	Name and role
Impact	Effect on scope, time, quality, risk
Decision	Approved, rejected, or deferred
Status	Current state (open / in progress / closed)
Notes	Rationale, conditions, follow-ups

Example change log

ID	Change	Requested by	Status	Decision
CR-01	Add Dark Mode	Team	Approved	Next sprint
CR-02	Add Chatbot	Client	Rejected	Out of scope
CR-03	UI color change	PO	Approved	Immediate

Chapter 8: Quality Management

8.1 Quality in Projects

In everyday language, “quality” often means luxury or high price. In project management, the meaning is different and more practical. Quality means whether the product or service meets the needs of its users and matches what was promised. Two classic definitions from quality experts help clarify this:

1. **Conformance to Requirements (Crosby, 1979)**

This means the project’s deliverables match the specifications that were agreed upon before work began. If the requirements document says the guidebook must have 10 chapters and 36 user stories, then a guidebook with exactly 10 chapters and 36 user stories conforms to requirements.

Simple example: You order a black coffee. You receive a black coffee. That is conformance.

2. **Fitness for Use (Juran & Godfrey, 1999)**

This means the deliverable works for the user in the real world. The guidebook may have all 10 chapters, but if a project manager cannot find the section on risk management quickly, or if the examples are confusing, then it is not fit for use.

Simple example: You order a black coffee. You receive black coffee, but it is cold and tastes burnt. It conforms to the order but is not fit for use.

8.2 Quality Planning and Control

Definitions of the Three Quality Management Processes

Quality Process	Simple Definition	Key Purpose
Quality Planning	Defines **what quality means**, the standards to meet, and how to measure them.	Establishes quality requirements, metrics, and the plan to achieve them.
Quality Assurance (QA)	Ensure the **processes** used in the project will produce the desired quality.	Prevents defects by validating that correct methods, standards, and procedures are being followed.
Quality Control (QC)	Inspects **deliverables** to verify they meet defined standards.	Detects defects through testing, measuring, and corrective action.

Actionable Software Development Examples

Quality Planning — "What quality should look like?"

Industry sources emphasize defining standards, risks, metrics, and documentation early.

Examples:

1. **Define coding standards and acceptance criteria**
   – e.g., establishing linting rules, Definition of Done, OWASP security compliance.
2. **Determine testing strategy**
   – setting required unit test coverage, required performance thresholds, and accessibility criteria.
3. **Create a quality metrics baseline**
   – defect density targets, acceptable error rates, and SLAs for response times.

Quality Assurance — "How do we ensure processes produce quality?"

QA is prevention-focused and ensures the team follows defined quality methods.

Examples:

1. **Conduct regular process audits**
   – reviewing sprint ceremonies, code review compliance, documentation completeness.
2. **Implement pair programming or peer reviews**
   – ensuring code quality before integration.
3. **Improve workflows based on retrospectives**
   – adjusting CI/CD pipelines, updating test strategies, improving definition of ready (DoR).

Quality Control — "Do the deliverables meet the standards?"

QC verifies outputs using tools like inspections, testing, and control charts.

Examples:

1. **Run automated test suites**
   – integration tests, regression tests, UI snapshots to detect errors.
2. **Defect identification and triage**
   – logging issues, prioritizing fixes based on severity.
3. **Manual product inspection / exploratory testing**
   – testers perform scenario-based tests to validate UX and functionality.

Agile vs. Waterfall: How Quality Management Differs

Modern research highlights major structural differences:
Agile = iterative, continuous feedback, and testing.
Waterfall = sequential, phase-gate quality checkpoints.

Quality Planning Differences

Waterfall

- Planning happens **upfront**, before development begins.
- Quality standards, test plans, and documentation must be fully defined before execution.
- Change is costly, as deviation requires revisiting earlier completed phases.

Agile

- Quality planning happens **iteratively**, revisited every sprint.
- Teams refine acceptance criteria and testing approaches continuously.
- Change is welcomed; planning adjusts to evolving requirements.

Quality Assurance Differences

Waterfall

- QA occurs **after planning** and often before testing phases.
- Process audits are done at phase-gates.
- A documentation-heavy approach ensures compliance before moving forward.

Agile

- QA is **embedded into everyday work activities**.
- Practices include continuous integration, peer reviews, and daily collaboration.
- Retrospectives drive continuous process improvement.

Quality Control Differences

Waterfall

- QC occurs **late in the lifecycle**, during testing after implementation.
- Defects are found after large chunks of work are completed.
- Testing is typically a distinct, dedicated phase.

Agile

- QC happens **continuously**: automated tests, sprint reviews, acceptance checks.
- Small increments ensure defects are caught early.
- Testing is integrated into each sprint and is not a single big phase.

Summary

Aspect	Agile	Waterfall
Quality Planning	Iterative, redefined each sprint	Upfront, rigid, and documented
Quality Assurance	Continuous (CI/CD, peer reviews)	Gate-based process audits
Quality Control	Ongoing testing throughout sprints	Large testing phase near the end
Risk of Defects	Low (early detection)	Higher (late discovery)
Adaptability	High	Low

Consequences of Poor-Quality Management

- **Rework and Delays:** Defects and errors require additional fixes, leading to missed deadlines and wasted effort.
- **Customer Dissatisfaction:** Bugs, poor performance, or unreliable features reduce user trust and satisfaction.
- **Increased Costs:** Fixing issues late or after release is more expensive than preventing them early.

Business Impact of Quality

- **Cost Impact:** High quality reduces rework, support needs, and unexpected expenses, while poor quality increases operational costs.
- **Reputation Impact:** Reliable products build customer trust, whereas major failures can seriously damage the organization's image.
- **Team Morale Impact:** Delivering quality work boosts confidence and motivation, while constant defects lead to stress and burnout.

How Quality Management Links to Project Success Metrics

Project Metric	Connection to Quality
ROI (Return on Investment)	Fewer defects → less rework → lower costs → higher ROI.
Stakeholder Satisfaction	High-quality deliverables meet expectations and build trust.
Time-to-Market	Good quality reduces last-minute issues that delay release.
Scope Delivery	Stable processes help teams deliver planned features without chaos.
Product Adoption	Users adopt and recommend a product that is reliable and performs well.

Quality isn't just a "nice to have"—it **directly drives business success**.

Real-World Example of Quality Leading to Project Success

Case: Microsoft Teams Performance Improvements (2020–2022)
Microsoft invested heavily in improving code quality, performance testing, and architecture optimization for Microsoft Teams.
This included:

- rewriting parts of the client for better memory usage
- implementing strict performance metrics
- expanding automated testing and regression detection

Outcome:

- App launch time improved for millions of users.
- Customer satisfaction scores increased.
- Teams' adoption grew globally, becoming a core tool during the remote-work boom.

This success was directly tied to a strong focus on quality assurance and performance metrics, showing how quality management drives both user satisfaction and business growth.

8.3 Quality tools and technique

Quality tools help project teams **prevent problems,** analyze **issues, and ensure deliverables meet expectations**.
The following section introduces **6 essential quality tools** commonly used in project management, along with simple explanations and practical project-level examples.

Checklists

Purpose

To ensure that important steps, approvals, or quality criteria are completed consistently.

When to Use

- During project phase handovers
- Before major deliverables are submitted to stakeholders
- During audits and compliance checks

Basic Steps

1. Identify key requirements or mandatory actions.
2. Create a structured list.
3. Review and tick off each item during execution.

Practical Project Example

Before closing a project phase, the PM uses a **Phase Closure Checklist** including:

- "All deliverables reviewed by stakeholders."
- "Risks updated"
- "Budget status documented."

This reduces the chance of missing critical tasks.

Root Cause Analysis (5 Whys)

Purpose

To determine the *real cause* behind a project issue, not just the visible symptom.

When to Use

- During lessons learned sessions
- When repeated issues affect schedule or deliverables
- After a major project delay

Basic Steps

1. State the problem clearly.
2. Ask "Why?" repeatedly (usually five times).
3. Stop when the underlying cause becomes clear

Practical Project Example

Problem: A deliverable was delivered late.

- Why? The team started the task late.
- Why? Requirements were not approved on time.
- Why? The sponsor was not available.
- Why? The approval meeting was not scheduled early enough.

Root Cause: Poor scheduling of stakeholder interactions.

Risk Analysis Matrix

Purpose

To evaluate risks by their likelihood and impact, helping teams prioritize what matters most.

When to Use

- During risk planning
- At the start of every major project phase
- In weekly or monthly status meetings

Basic Steps

1. Identify risks.
2. Rate each risk's probability and impact.
3. Place them on the matrix (Low / Medium / High).
4. Plan mitigation strategies.

Practical Project Example

A high-impact, high-probability risk such as "Vendor delay" is flagged as red, prompting proactive mitigation (e.g., backup vendors).

Cause-and-Effect Diagram

Purpose

To visually identify all possible causes of a project problem by grouping them into categories.

When to Use

- When a complex issue occurs
- During team workshops
- To investigate quality failures in deliverables

Basic Steps

1. Write the problem at the head of the diagram.
2. Draw major branches (e.g., People, Process, Tools, Environment).
3. Brainstorm possible causes under each branch.

Practical Project Example

For "Low stakeholder satisfaction," possible causes identified include:

- **People:** unclear roles
- **Process:** slow communication
- **Tools:** inconsistent reporting
- **Environment:** shifting priorities

This helps the team focus efforts on the right area.

Flowcharts

Purpose

To map how a process works, helping teams identify inefficiencies, gaps, or unnecessary steps.

When to Use

- When designing project processes (e.g., change requests)
- During process improvement initiatives
- When onboarding new team members

Basic Steps

1. Identify the start and end points.
2. Map each step in sequence.
3. Highlight decision points.
4. Analyze bottlenecks or problems.

Practical Project Example

A PM maps the change request process and discovers approval passes through too many managers. Streamlining reduces turnaround time.

Lessons Learned Register

Purpose

To capture insights, mistakes, and successes for continuous improvement.

When to Use

- End of each phase
- During sprint retrospectives
- At project closure

Basic Steps

1. Record the event or issue.
2. Document what went well or poorly.
3. Identify recommended actions.
4. Share with the wider organization.

Practical Project Example

A project experiences repeated miscommunication with a supplier.
Lessons learned: “Set weekly check-ins with all external partners.”
Future projects adopt this and avoid similar problems.

Quick-Reference Guide: Choosing the Right Quality Tool

Tool	Best Use Case	Primary Benefit
Checklists	Phase transitions, deliverable reviews	Ensures consistency and avoids missed tasks
Root Cause Analysis (5 Whys)	Repeating problems or major failures	Identifies the true underlying cause
Risk Analysis Matrix	Risk planning & prioritization	Helps focus on high-impact, high-probability risks
Fishbone Diagram	Complex quality issues	Break down all potential causes
Flowcharts	Process mapping & improvement	Reveals inefficiencies and gaps
Lessons Learned Register	End of phases or projects	Drives continuous improvement

Chapter 9: PM Software

9.1 Popular Project Management tools

In the software industry, tools like Jira, Trello, and Microsoft Planner are widely used for managing projects. These tools help teams plan tasks, track progress, and collaborate effectively. Each tool supports different project management styles and team needs.

Short Description of Each Tool

Jira: Jira is mainly used by software development teams. It is designed to support Agile methodologies such as Scrum and Kanban. It allows teams to track tasks, manage backlogs, and handle bug tracking efficiently. Jira is highly customizable, which makes it suitable for complex and long-term projects.

Trello: Trello is a simple and visual project management tool. It uses boards, lists, and cards to organize tasks. It is easy to learn and use, which makes it ideal for beginners or small teams. However, it has limited advanced features compared to Jira.

Microsoft Planner: Microsoft Project Planner is a professional tool used for detailed project planning and management. It focuses on scheduling, resource allocation, and timeline tracking. It is commonly used in large organizations and supports traditional project management methods like Waterfall.

Pricing Comparison Table

Tool	Free Plan	Starting Price	Notes
Jira	Yes (up to 10 users)	$7.91 user/month	Flexible and scalable, but can become complex for beginners
Trello	Yes	$5.00 per user/month	Low cost and simple, but limited advanced features
Microsoft Planner	No (trial only)	$10.00 per user/month	Powerful but expensive and requires learning

Comparison and Suitability

These tools differ mainly in their methodology support and level of complexity:
Jira supports Agile methodologies and is best for software development teams working on medium-to-large projects. It provides strong tracking and reporting features.
Trello supports basic Kanban-style task management and is best for small teams or simple projects where ease of use is important.
Microsoft Project Planner supports traditional (Waterfall) project management and is best for large-scale projects that require detailed planning and resource management.

Why are these tools widely used?

These tools are widely adopted because they improve productivity, organization, and collaboration in teams. They allow better tracking of tasks, deadlines, and responsibilities. From a critical point of view, each tool has its strengths and limitations. Jira is powerful but can be complex, Trello is simple but limited, and Microsoft Project is detailed but costly and harder to use.
Because different projects have different requirements, companies choose tools based on their project size, complexity, and working methodology. This flexibility is the main reason why these tools are popular in the software industry.

9.2 Choosing the right software

Before choosing a project management tool, you need to know what features matter for your project. Different tools suit different team sizes, methodologies, and budgets. This section helps you understand common features and how to match them with your needs.

What to Look for in a PM Tool

When choosing a project management tool, focus on these key features:

1. **Task Management:** Create, assign, and track tasks. Set due dates and mark progress.
2. **Collaboration:** Comment on tasks, tag team members, attach files, and receive notifications.
3. **Progress Visibility:** See what the team is working on and what is completed using boards, lists, or timelines.
4. **Backlog Management:** Maintain a prioritized list of work items for future sprints or phases.
5. **Reporting:** Generate simple reports on progress, completed work, and upcoming deadlines.

Features Needed for Different Methodologies

Methodology	Must-Have Features
Scrum / Agile	Backlog, sprint planning, Kanban board, burndown chart
Waterfall	Gantt chart, task dependencies, milestones, resource tracking
Hybrid	Both backlog and Gantt capabilities

How to Choose the Right Tool

Follow these steps when selecting a tool:

- **Step 1: Know your team size:** Small teams can use free tools; large teams may need paid plans.
- **Step 2: Know your methodology:** Scrum needs backlog and boards; Waterfall needs Gantt charts.
- **Step 3: Know your budget:** Many good tools have free tiers for small teams.
- **Step 4: Test before committing:** Try the free version first and get team feedback.

Quick Selection Guide

If you need...	Choose...
Simple task tracking for a small Scrum team	Trello
Full agile support with advanced reporting	Jira
Gantt charts and waterfall planning	MS Project
A balance of both agile and waterfall	Asana or ClickUp

Questions to Ask Before Choosing

- How many people will use this tool?
- Does it support our way of working (Scrum, Waterfall, or both)?
- Is there a free plan that meets our needs?
- How long will it take the team to learn?
- Does it work with our other tools?

Chapter 10: Professional Development

10.1 PM Associations & Communities

Joining a professional association is one of the most effective ways to stay current, build your network, and advance your project management career (PMI, 2021). This section provides an overview of the leading project management associations at global and regional levels, helping you decide which community best fits your professional goals.

Major International Associations

The following six associations are the most widely recognized globally. Each serves a distinct audience and focus area.

Association	Focus	Target Audience	Key Offerings	Membership Fee	Website	Best For
PMI	Traditional PM, all methods	All career stages, students to executives	PMP®, CAPM®, PMI-ACP®, PMBOK® Guide, local chapters	USD 129–169 / yr	pmi.org	Globally recognised PM credentials
Agile Alliance	Agile principles & practice	Agile practitioners across industries	Resource library, online events, Agile20xx conference	USD 59 / yr	agilealliance.org	Agile community & resources
Scrum Alliance	Scrum practice	Scrum Masters, POs, developers, coaches	CSM®, CSPO®, micro-credentials, Regional Scrum Gatherings	Included with cert; renewal USD 50–250 / 2 yrs	scrumalliance.org	Scrum certification track
IPMA	Competence-based PM	Project professionals in 70+ countries	4-level ICB certification, World Congress, Member Associations	Varies (e.g. EUR 125 / yr in Austria)	ipma.world	Europe-focused, competence-based path
APM	Chartered UK profession	UK and international professionals, students to fellows	APM qualifications, APM Learning, mentoring, Project journal	Free for students	apm.org.uk	UK-based professionals & students
IIBA	Business analysis	Business analysts, requirements professionals	CBAP®, CCBA®, ECBA™, global chapters	Varies by region	iiba.org	BA-focused careers

Regional and National Associations (Example: Finland)

For professionals who prefer local networking and events in their own language and time zone, national associations are excellent resources.
Many associations offer free or heavily discounted student memberships (e.g., APM free student membership, PMI reduced rates for students). Take advantage of these while you are still studying.

Association	Description	Website
PMI Finland Chapter	Non-profit, volunteer-run chapter with 300+ members since 2005. Organises "Passion for Projects", Scandinavia's largest meeting place for project professionals.	pmi-fi.org
Projektiammattilaiset PRY	The largest project association in Finland with nearly 3,500 members. Offers networking, benchmarking, knowledge sharing, and career support across all industries.	pry.fi

Comparison of Membership Benefits

This table compares concrete benefits across the five major associations to help you weigh trade-offs before joining.

Benefit	PMI	Agile Alliance	Scrum Alliance	IPMA	APM
Certifications	PMP, CAPM, PMI-ACP	None (resource-focused)	CSM, CSPO	4-level ICB	APM PFQ, PMQ
Exam fee discount	Yes (USD 150 off PMP)	N/A	Included with training	Yes	Yes
Library / standards	PMBOK®, Infinity™	Sessions, reports, videos	Member-only resources	ICB	APM Learning
Local chapters	300+ globally	300+ community groups	Regional Gatherings	70+ Member Associations	Regional networks
Annual conference	PMI Global Summit	Agile20xx	Global Scrum Gathering	IPMA World Congress	APM PM Conference
Networking	Mentorship, volunteering	Meet-ups, Lean Coffee	User groups	Federation events	Mentoring programme
CPD credits	Yes (PDUs)	N/A	Yes (SEUs)	Yes	Yes (CPD)
Student membership	Reduced rates	No	No	Varies by MA	Yes (free)
Post-nominals	PMP, CAPM, etc.	No	CSM, CSPO	IPMA Level D–A	MAPM, FAPM

How Joining an Association Supports Career Growth

Professional associations contribute to long-term career development in six concrete ways:

Benefit	How It Supports Your Career
Continuous learning	Access to current standards (e.g., PMBOK® Guide), research, webinars, and e-learning. Keeps you current as practices evolve, including emerging topics like AI in PM.
Networking & mentorship	Local chapter events, online communities, and mentoring programmes connect you with experienced practitioners who share lessons and refer opportunities.
Professional credibility	Certifications and post-nominals (PMP, MAPM) signal to employers that you have met rigorous standards. PMI-certified professionals earn significantly higher median salaries (PMI, 2023).
Volunteer leadership	Chapters are run by volunteers. Volunteering offers low-risk opportunities to develop leadership, event-planning, and governance skills while contributing to the community.
Job market access	Most associations maintain job boards tailored to PM roles, helping you find positions matching your certification and experience.
Demonstrated commitment	Maintaining membership and earning continuing education credits (PDUs, SEUs, CPD) signals to employers that you are serious about professional growth.

How to Get Started

Step	Action	Detail
1	Identify your focus area	Traditional PM → PMI or IPMA. Agile/Scrum → Agile Alliance or Scrum Alliance. UK-based → APM. Business analysis → IIBA.
2	Check student options	If you are a student, look for free or discounted memberships (e.g., APM free student membership).
3	Join one association first	Start with one that aligns with your current role or career goal. You can always add more later.
4	Attend a local event	Most chapters welcome non-members to their first event. Attend to see if the community fits.
5	Volunteer early	Even small volunteer roles (e.g., helping at a workshop) build your network and CV.

10.2 Certifications

Professional certifications validate your knowledge, signal commitment to the field, and have a measurable impact on career prospects. This section reviews the six most recognized PM certifications, compares them across key dimensions, and offers guidance on which to pursue first (PMI, 2023).

Overview of Key Certifications

Six certifications dominate the global PM market. Each targets a different audience and methodology.

Certification	Issuer	Target Audience	Eligibility	Best For
PMP — Project Management Professional	PMI	Experienced PMs leading projects in any methodology	4-yr degree + 36 mo PM experience + 35 hrs training (or HS diploma + 60 mo experience)	Generalist PMs across industries; the global gold standard
CSM — Certified ScrumMaster®	Scrum Alliance	Professionals new to Agile/Scrum	Mandatory 2-day course (14–16 hrs) by a Certified Scrum Trainer	Entry-level Scrum Masters who prefer structured training
PSM — Professional Scrum Master™	Scrum.org	Scrum Masters and Agile coaches at any level	No prerequisites; training recommended but not required	Rigorous self-paced Scrum assessment without mandatory courses
PMI-ACP — Agile Certified Practitioner	PMI	Experienced Agile practitioners across frameworks	Secondary degree + 24 mo Agile experience + 28 hrs Agile training	Multi-framework Agile (Scrum, Kanban, Lean, XP)
SAFe® Agilist (SA)	Scaled Agile	Senior leaders driving Agile transformation at scale	No formal prerequisites; 5+ yrs PM/dev experience strongly recommended	Enterprise-scale Agile transformations
PRINCE2® Agile	PeopleCert	PMs in organisations needing formal governance + Agile delivery	Foundation: none. Practitioner: PRINCE2 Foundation required	UK / Europe / Australia where PRINCE2 is standard

Comparison of Certifications

Certification	Focus Area	Primary Methodology	Experience Level	Global Recognition
PMP	Traditional + Agile + Hybrid	Methodology-agnostic	Mid to senior	Highest (global)
CSM	Scrum team leadership	Scrum only	Entry to mid	High (US/Europe)
PSM	Deep Scrum mastery	Scrum only	Entry to senior	High (global)
PMI-ACP	Multi-method Agile	Scrum, Kanban, Lean, XP	Experienced	High (global)
SAFe Agilist	Enterprise Agile scaling	SAFe	Senior	Growing (large enterprises)
PRINCE2 Agile	Governance + Agile	PRINCE2 + Agile	Mid to senior	High (UK, Europe, Australia)

Exam Format, Preparation, and Renewal

Certification	Typical Prep	Duration	Questions	Pass Score	Renewal
PMP	2–3 months (60–80 hrs)	230 min	180	Not disclosed	60 PDUs / 3 yrs
CSM	2-day mandatory course	60 min	50	74%	20 SEUs / 2 yrs + USD 100
PSM I	Self-study (1–4 weeks)	60 min	80	85%	None — does not expire
PMI-ACP	1–2 months (40–60 hrs)	180 min	120	~70% (estimated)	30 PDUs / 3 yrs
SAFe Agilist	2-day course + self-study	90 min	45	80%	Annual fee
PRINCE2 Agile (Foundation)	2–3 days training	60 min	50	55%	None at Foundation

Salary Impact and Career Pathway

Certifications correlate strongly with higher compensation. The figures below are drawn from PMI's 14th Salary Survey and PayScale (PMI, 2023; PayScale, 2024).

Certification	Salary Indicator
PMP (global)	17% higher median salary than non-certified peers across 21 countries
PMP (United States)	USD 135,000 median salary — 24% higher than non-certified peers
PMP (annual raise)	Nearly 60% of holders reported a salary increase in the past year
CSM (UK)	~GBP 56,000 median annual salary
PMI-ACP (UK)	~GBP 50,000 median annual salary

Which Certification Should You Pursue First?

PMI projects a global need for 30 million project professionals by 2035 (PMI, 2023). For most newcomers, the recommended progression is:

Step	Certification	Why First
1	CSM or PSM I	Quick to obtain; high immediate value for team-level Agile roles
2	PMP	Career-defining; validates broad PM experience and unlocks the largest salary impact
3	Specialize	Agile depth → PMI-ACP or PSM II/III. Scaling → SAFe Agilist. Governance → PRINCE2 Agile Practitioner

Chapter 11: Appendices

11.1 Source references

Association for Project Management. (n.d.). *Association for Project Management (APM)*. https://www.apm.org.uk/

Atlassian. (n.d.-a). *Backlog refinement*. https://www.atlassian.com/agile/scrum/backlog-refinement

Atlassian. (n.d.-b). *Prioritization*. https://www.atlassian.com/agile/product-management/prioritization

Atlassian. (n.d.-c). *Scrum backlog*. https://www.atlassian.com/agile/scrum/backlogs

Atlassian. (n.d.-d). *User stories*. https://www.atlassian.com/agile/user-stories

Beck, K. (2000). *Extreme programming explained: Embrace change*. Addison-Wesley.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. https://agilemanifesto.org/

Boehm, B., & Turner, R. (2003). *Balancing agility and discipline: A guide for the perplexed*. Addison-Wesley.

Crosby, P. B. (1979). *Quality is free: The art of making quality certain*. McGraw-Hill.

Edraw. (n.d.). *Software development project Gantt chart template*. https://www.edrawsoft.com/template-develop-new-software-gantt-chart.html

Gleek. (n.d.). *Gantt charts and the critical path method*. https://www.gleek.io/blog/gantt-critical-path

Highsmith, J. (2009). *Agile project management: Creating innovative products* (2nd ed.). Addison-Wesley.

International Institute of Business Analysis. (n.d.). *IIBA*. https://www.iiba.org/

International Project Management Association. (n.d.). *IPMA*. https://ipma.world/

Juran, J. M., & Godfrey, A. B. (1999). *Juran's quality handbook* (5th ed.). McGraw-Hill.

Leffingwell, D. (2007). *Scaling software agility: Best practices for large enterprises*. Addison-Wesley.

Mountain Goat Software. (n.d.). *User stories*. https://www.mountaingoatsoftware.com/agile/user-stories

Palmer, S. R., & Felsing, J. M. (2002). *A practical guide to feature-driven development*. Prentice Hall.

PMI Finland Chapter. (n.d.). *PMI Finland Chapter*. https://pmi-fi.org/

Project Management Institute. (n.d.). *Project Management Institute*. https://www.pmi.org/

Project Management Institute. (2017). *Agile practice guide*.

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK guide)* (7th ed.).

Project Management Institute. (2023). *Earning power: Project management salary survey* (13th ed.).

Project Management Institute. (2025). *Global project management talent gap report*. https://www.pmi.org/learning/thought-leadership/global-project-management-talent-gap

Projektiammattilaiset PRY. (n.d.). *Project Professionals Finland (PRY)*. https://www.pry.fi/

Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON, 26*, 1–9.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. https://scrumguides.org/

Scrum Alliance. (n.d.). *Scrum Alliance*. https://www.scrumalliance.org/

Scrum.org. (n.d.-a). *Ordering the product backlog*. https://www.scrum.org/resources/ordering-product-backlog

Scrum.org. (n.d.-b). *Product backlog refinement: How to succeed*. https://www.scrum.org/resources/blog/product-backlog-refinement-how-succeed-scrum-team

Scrum.org. (n.d.-c). *Resources*. https://www.scrum.org/resources

Scrum.org. (n.d.-d). *What is a product backlog?* https://www.scrum.org/resources/what-is-a-product-backlog

ScrumStudy. (n.d.-a). *Product backlog management*. https://www.scrumstudy.com/article/product-backlog-management

ScrumStudy. (n.d.-b). *Scrum agile product backlog*. https://www.scrumstudy.com/article/scrum-agile-product-backlog

Smartsheet. (n.d.). *Free Gantt chart templates with dependencies*. https://www.smartsheet.com/content/gantt-chart-templates-dependencies

11.2 Ethical AI use statement

Guidelines for AI Use

Allowed	Not Allowed
Brainstorming ideas and improving structure	Copying AI output without review
Checking grammar and refining language	Generating complete answers for assignments
Summarizing information for understanding	Bypassing learning or critical thinking

Core Principle: AI is a tool to assist learning, not replace it. All AI-assisted content must be reviewed and verified by the student.

How AI Was Used in This Guidebook

Use Case	How AI Was Used
Idea generation	Brainstorming chapter structures and key topics
Content structuring	Organizing information into clear sections
Language refinement	Improving clarity, grammar, and readability
Template creation	Generating reusable templates (chapter, self-assessment, style guide)
Research assistance	Summarizing reference materials like the IPMA ICB document

Important: All AI-generated suggestions were reviewed, edited, and approved by the student before inclusion. No AI output was copied without verification.

AI Usage Statement

AI tools were used as a supportive resource for brainstorming, structuring, language refinement, template creation, and research summarization. All AI-assisted content has been reviewed, validated, and aligned with course requirements by the student author, who takes full responsibility for the final work.

11.3 Ready-to-use templates

Template of Change Request Form

First page filled by the requester

Project name: _____
Requester name and contact info: _____
Role in project: _____
Request Date: _____
Request Priority:
HIGH ___ MEDIUM___ LOW___

Date the change is needed: _____

Change requested (for example, “Removal / creation of a chapter”):

_____
_____

Reason for this request:

_____
_____
_____

Detailed description of a wanted change:

_____
_____
_____
_____
_____
_____
_____

Second page filled by the team

Impact analysis:

Resources/Cost:

_____

_____

Deliverables:

_____

_____

Timeline:

_____

_____

Workload: _____

Scope:

_____

_____

_____

Risk Analysis:

Risk Identification:

_____

Probability of Risk: HIGH___MEDIUM___ LOW___

Risk Mitigation strategy:

_____

_____

_____

DECISION: ACCEPTED___ REJECTED____